COMO FAZER CADASTRO COM CHAVE PRIMÁRIA COM AUTOINCREMENTO

a) Alguns problemas que temos quando trabalhamos Java e Myql com chave primário com autoincremento:

1) **Não usar Statement.RETURN_GENERATED_KEYS**

Quando você insere um registro em uma tabela que tem AUTO_INCREMENT, o MySQL gera automaticamente o valor da chave primária. Se você não pedir explicitamente para o JDBC retornar essa chave, o Java não sabe qual foi o id gerado.

Exemplo errado:

```
PreparedStatement ps = conn.prepareStatement("INSERT INTO notas (id_cliente, data) VALUES (?, NOW())");
ps.setInt(1, idCliente);
ps.executeUpdate();
int idNota = ps.getGeneratedKeys().getInt(1); // ERRO: não pediu para retornar a chave
```

Exemplo correto:

```
PreparedStatement ps = conn.prepareStatement(
 "INSERT INTO notas (id_cliente, data) VALUES (?, NOW())",
 Statement.RETURN_GENERATED_KEYS // precisa disso!
);
ps.setInt(1, idCliente);
ps.executeUpdate();

ResultSet rs = ps.getGeneratedKeys();
if (rs.next()) {
 int idNota = rs.getInt(1); // aqui pega o AUTO_INCREMENT
 System.out.println("ID da nota gerado: " + idNota);
}
```

2. Tentar inserir manualmente o ID

Se sua tabela está com AUTO_INCREMENT, você não deve passar o valor da PK no INSERT.

Errado:

```
INSERT INTO clientes (id, nome) VALUES (10, 'João');
```

Certo:

```
INSERT INTO clientes (nome) VALUES ('João');
```

O MySQL cuida do campo id.

3. Tipo incorreto da coluna AUTO_INCREMENT

O campo precisa ser chave primária (PRIMARY KEY) e numérico (INT, BIGINT, etc.).
Se você colocou VARCHAR, CHAR ou esqueceu o PRIMARY KEY, o MySQL não gera corretamente o valor.

Exemplo correto:
id INT PRIMARY KEY AUTO_INCREMENT

4. Driver JDBC antigo

Versões antigas do mysql-connector-java (ex.: 5.x) tinham bugs com getGeneratedKeys().
O ideal é usar a versão mais recente (8.x).

5. Conflito de engine do MySQL

Se a tabela não estiver usando InnoDB ou MyISAM, pode dar erro com autoincremento (dependendo da versão/configuração).
O mais seguro é:

ENGINE=InnoDB

O que é a ENGINE no MySQL?
A engine (mecanismo de armazenamento) é a "camada interna" que o MySQL usa para gerenciar como os dados são armazenados, organizados e manipulados dentro das tabelas.
Ou seja:
Você escreve SQL (CREATE TABLE, INSERT, SELECT…),
Mas quem realmente decide como os dados vão parar no disco e como são acessados é a engine.
Pensa nela como o motor por baixo da tabela.

Principais engines do MySQL
- InnoDB (padrão)

Hoje é a engine padrão do MySQL.
Suporta:
Transações (BEGIN, COMMIT, ROLLBACK)
Chaves estrangeiras (FOREIGN KEY)
Integridade referencial (garante consistência entre tabelas)
Bloqueio em nível de linha (mais performance em ambientes concorrentes)
É a mais usada em sistemas reais.

MyISAM (antiga padrão)
Mais antiga, mais simples.
Não suporta transações nem chaves estrangeiras.
Só bloqueia a tabela inteira (menos eficiente).
Mais leve para leitura, mas perigosa para sistemas críticos.

Outras menos usadas
MEMORY → tabela fica na RAM, super rápida, mas some se desligar o servidor.
ARCHIVE → para armazenar grandes quantidades de dados históricos.
CSV → salva dados em formato .csv.

Quando você define a engine que vai usar no MySQL ?

Pode ser :
1) Quando você cria a tabela

Exemplo de criação já com InnoDB:

```
CREATE TABLE clientes (
 id INT PRIMARY KEY AUTO_INCREMENT,
 nome VARCHAR(100) NOT NULL,
 cpf VARCHAR(20)
) ENGINE=InnoDB;
```

2) Para alterar uma tabela existente

Se a tabela já existe mas está em outra engine (ex.: MyISAM), você pode trocar:
ALTER TABLE clientes ENGINE=InnoDB;

3) Para deixar como padrão no banco

Se quiser que todas as tabelas novas já usem InnoDB automaticamente, configure no MySQL:

SET default_storage_engine=InnoDB;

Ou no arquivo de configuração my.cnf (Linux) / my.ini (Windows):

[mysqld]
default-storage-engine = InnoDB
Depois, reinicia o MySQL.

b) Cadastrar cliente com o ID autoincrement

```
PreparedStatement ps = conn.prepareStatement(
                "INSERT INTO clientes (nome, cpf) VALUES (?, ?)",
                Statement.RETURN_GENERATED_KEYS
        );
        ps.setString(1, nome);
        ps.setString(2, cpf);
        ps.executeUpdate();

        ResultSet rs = ps.getGeneratedKeys();
        if (rs.next()) {
           int idGerado = rs.getInt(1);
           JOptionPane.showMessageDialog(this, "Cliente salvo! ID gerado: " + idGerado);
        }
```

c) Pesquisar um cliente pelo nome

```
private void buscarCliente() {
        String nome = txtCliente.getText();
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/sistema",
"root", "senha");
             PreparedStatement ps = conn.prepareStatement("SELECT id, nome FROM clientes
WHERE nome LIKE ?")) {

            ps.setString(1, "%" + nome + "%");
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                idClienteSelecionado = rs.getInt("id");
                lblClienteSelecionado.setText("Cliente: " + rs.getString("nome"));
            } else {
                JOptionPane.showMessageDialog(this, "Cliente não encontrado.");
            }
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
```

d) Inserir dados em uma Table

```
modeloTabela = new DefaultTableModel(new String[]{"Produto", "Qtd", "Valor Unitário",
"Total"}, 0);

private void adicionarItem() {
        JTextField txtProduto = new JTextField();
        JTextField txtQtd = new JTextField();
        JTextField txtPreco = new JTextField();

        Object[] message = {
            "Produto:", txtProduto,
            "Quantidade:", txtQtd,
            "Preço Unitário:", txtPreco
        };

        int option = JOptionPane.showConfirmDialog(this, message, "Adicionar Item",
JOptionPane.OK_CANCEL_OPTION);
        if (option == JOptionPane.OK_OPTION) {
            String produto = txtProduto.getText();
            int qtd = Integer.parseInt(txtQtd.getText());
            double preco = Double.parseDouble(txtPreco.getText());
            double total = qtd * preco;

            modeloTabela.addRow(new Object[]{produto, qtd, preco, total});
        }
    }
```